

System Design Cheat Sheet

The Essential Quick-Reference Guide for Interview Day

FREE EDITION · By Shekhar Patil · © 2026

RESHADED Framework — Use This in Every Interview

Letter	Step	Time	What To Say
R	Requirements	5 min	"What are the 2–3 core features? Who are the users? Read or write heavy?"
E	Estimation	5 min	" $150\text{M DAU} \times 100 \text{ reads/day} \div 86,400 = \sim 175\text{K reads/sec}$ "
S	Storage	5 min	"User data → PostgreSQL. Activity logs → Cassandra. Media → S3."
H	High-level	10 min	"Client → CDN → LB → API Servers → Cache → DB"
A	APIs	5 min	"POST /tweets, GET /timeline/{user_id}?limit=20&cursor;=..."
D	Deep Dive	10 min	"Let me go deep on the feed generation — the hardest part."
E	Edge Cases	3 min	"What if a celebrity has 50M followers? Use hybrid push/pull."
D	Decisions	2 min	"I chose Cassandra here because writes dominate — here's why."

Numbers Every Engineer Must Know Cold

Operation	Latency	Remember As
L1 cache read	0.5 ns	Instant
RAM read	100 ns	200× L1
SSD random read	100 μs	1,000× RAM
HDD seek	10 ms	10M× L1
Intra-DC RTT	0.5 ms	Same building
Cross-continent RTT	150 ms	Speed-of-light limit
1M req/day → QPS	~12 req/s	÷ 86,400
10M req/day → QPS	~115 req/s	÷ 86,400
99.9% uptime	8.7 hrs downtime/yr	Three 9s
99.99% uptime	52 min/yr	Four 9s (needs redundancy)

Availability Patterns

Pattern	When To Use	Example
Active-Active	High availability; both serve traffic	2 DB masters, both accept writes
Active-Passive	Failover; standby promoted on failure	MySQL primary + warm standby

Pattern	When To Use	Example
N+1 Redundancy	N servers needed; keep 1 spare ready	5 app servers, 1 on standby
Geographic redundancy	Disaster recovery; serve from nearest region	Multi-region AWS deployment

Technology Selection Guide

Use Case	Technology	Why
Relational data, ACID, complex queries	PostgreSQL	Powerful, proven, open-source
Cache, sub-ms reads, pub/sub	Redis	In-memory, ~1ms, rich data structures
Write-heavy, time-series, IoT	Cassandra	Append-only LSM, scales writes linearly
Full-text / relevance search	Elasticsearch	Inverted index, near real-time
Event streaming, audit log	Kafka	Durable, replayable, millions/sec
Object storage (media, files)	AWS S3	Infinite scale, 11-nines durability
Global key-value, serverless	DynamoDB	Auto-scale, single-digit ms latency
Distributed locks, consensus	etcd / ZooKeeper	Raft consensus, leader election
Graph relationships, social	Neo4j	Native graph DB, Cypher queries

Cache Strategies at a Glance

Strategy	How	Best For	Watch Out
Cache-Aside	App checks cache; miss → DB → write cache	General purpose (most common)	Cold start miss; stale if DB updated
Write-Through	Write cache + DB together	Read-heavy; freshness critical	Slower writes; caches rarely-read data
Write-Behind	Write cache; async flush to DB	Ultra-low write latency	Data loss if cache crashes before flush
Read-Through	Cache fetches from DB on miss	Transparent to app	Cache layer must know DB schema

CAP Theorem Decision Tree

Question	Choose	Examples
Is correctness critical? (banking, inventory)	CP — Consistency + Partition Tolerance	HBase, ZooKeeper, MongoDB (strong)
Is uptime critical? (social, shopping cart)	AP — Availability + Partition Tolerance	Cassandra, DynamoDB, CouchDB
Single node, no distribution needed?	CA — Consistency + Availability	MySQL, Postgres (single-node)
Default choice for most interview systems?	AP with eventual consistency	Cassandra + read repair + vector clocks

Rate Limiting Algorithms

Algorithm	How It Works	Pros	Cons
Token Bucket	Tokens refill at rate R; each request consumes 1	Allows bursts; simple	Burst can spike if bucket is full
Leaky Bucket	Process requests at fixed rate; overflow dropped	Smooth output; predictable	Bursts queued or dropped
Fixed Window	Count per time window (e.g., per minute); reset at boundary	Easiest to implement	Boundary attack doubles limit
Sliding Window	Track timestamps; count requests in last N seconds	Very accurate	Memory-intensive at high traffic
Sliding Window Counter	Weighted combination of current + previous window	Accurate + memory efficient	Approximate

Common Design Patterns

Pattern	Problem It Solves	How
Circuit Breaker	Cascading failure when downstream service is down	Open circuit after N failures; fail fast; probe to recover
Saga	Distributed transaction across microservices	Chain of local transactions; compensating rollback on failure
CQRS	Read and write models have different scaling needs	Separate read DB (optimised for queries) from write DB
Outbox Pattern	Reliable event publishing after DB write	Write event to outbox table atomically; separate publisher reads and sends
Strangler Fig	Migrate from monolith to microservices safely	Route new traffic to microservice; gradually strangle the monolith
Sidecar	Cross-cutting concerns (logging, auth, tracing) per service	Deploy sidecar container alongside each service (Envoy, Linkerd)

Case Study Quick Reference

System	Key Numbers	Core Challenge	Key Tech Choice
URL Shortener	1K writes/sec, 100K reads/sec	ID generation (no collisions), fast redirect	Base62 encoding, Redis for redirect cache, MySQL
Twitter Feed	3,500 tweets/sec, 175K reads/sec	Fan-out to followers at scale	Hybrid push/pull: Redis timelines + Cassandra tweets
WhatsApp	1.2M msgs/sec, 2B users	Real-time delivery + offline buffering	WebSocket, Erlang BEAM, Kafka, E2E encryption
Netflix	1B hrs/day, 15% internet BW	Adaptive streaming globally	ABR, Open Connect CDN, S3 for video chunks
Uber	19M trips/day, sub-2s matching	Real-time geospatial matching	Redis GEO + Geohash, ETA-based ranking, Kafka state machine
Google Drive	1B users, petabytes	Sync + dedup + conflict resolution	Chunked upload, SHA-256 dedup, CRDT for collaboration

The 10 Golden Rules

- | | |
|--|---|
| 1. Clarify before designing | 6. Async everything non-critical |
| 2. Estimate first, design second | 7. Design for failure — every component will fail |
| 3. Start simple, earn complexity | 8. State trade-offs, not just decisions |
| 4. Stateless services scale | 9. Horizontal scaling > vertical scaling |
| 5. Cache the 20% that handles 80% of traffic | 10. Monitor, alert, iterate |

System Design: The Complete Interview Masterclass

© 2026 Shekhar Patil. Free for personal use. Redistribution without permission is prohibited.